parisons. In addition, we find that results generally follow a trend based upon model size. Our results are in accord with previous work such as Wallace et al. (2019); However, we have also highlighted further model weaknesses through our probing methods. Therefore, the opportunities for future work, especially with a multilingual focus, are plenty.

Acknowledgements

Thank you to all anonymous reviewers for your helpful comments. Thank you to Professor Shane Steinert-Threlkeld for your guidance and throughout all stages of this paper. Thank you to Professor Luke Zettlemoyer for your advice on our paper in its earlier stages. Thank you to the several volunteer native speakers for grammaticality judgments. –This work was facilitated through the use of advanced computational, storage, and networking infrastructure provided by the Hyak supercomputer system and funded by the STF at the University of Washington.–

References

Yonatan Belinkov and James R. Glass. 2019. Analysis methods in neural language processing: A survey. *Transactions of the Association for Computational Linguistics*, 7:49–72.

J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. 2018. Bert: Pre-training of deep bidirectional transformers for language understanding.

Allyson Ettinger. 2020. What bert is not: Lessons from a new suite of psycholinguistic diagnostics for language models. *Transactions of the Association for Computational Linguistics*, 8:34–48.

Allyson Ettinger, Ahmed Elgohary, and Philip Resnik. 2016. Probing for semantic evidence of composition by means of simple classification tasks. In *Proceedings of the 1st Workshop on Evaluating Vector-Space Representations for NLP*, pages 134–139, Berlin, Germany. Association for Computational Linguistics.

Guillaume Lample and Alexis Conneau. 2019. Cross-lingual language model pretraining. *CoRR*, abs/1901.07291.

K. F. Miller, C. M. Smith, J. Zhu, and H. Zhang. 1995. Preschool origins of cross-national differences in mathematical competence: The role of number-naming systems. *Psychological Science*, 6:56–60.

T. Ogawa. num2words.

Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2020. Distilbert, a distilled version of bert: smaller, faster, cheaper and lighter.

Eric Wallace, Yizhong Wang, Sujian Li, Sameer Singh, and Matt Gardner. 2019. Do nlp models know numbers? probing numeracy in embeddings. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*.

Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, R'emi Louf, Morgan Funtowicz, and Jamie Brew. 2019. Huggingface's transformers: State-of-the-art natural language processing. *ArXiv*, abs/1910.03771.

A Fine-Tuning

The fine-tuning results shown in (Figure 3) are validation accuracies after one epoch of training. When trained for 20 epochs (Figure 4), the models reach over 99% accuracy on all languages and tasks except for Danish which reaches 95% on sentence tasks.

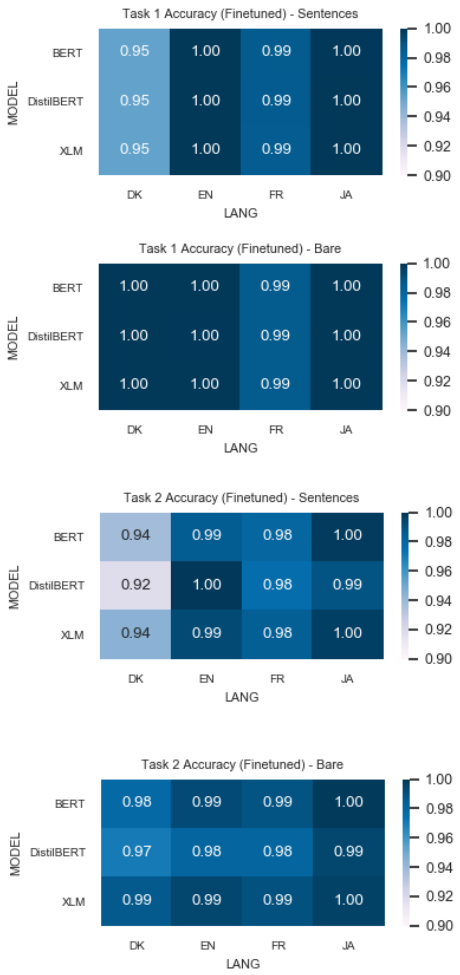

Figure 3: Fine-tuned task validation accuracy per language/model, when trained for 1 epoch.

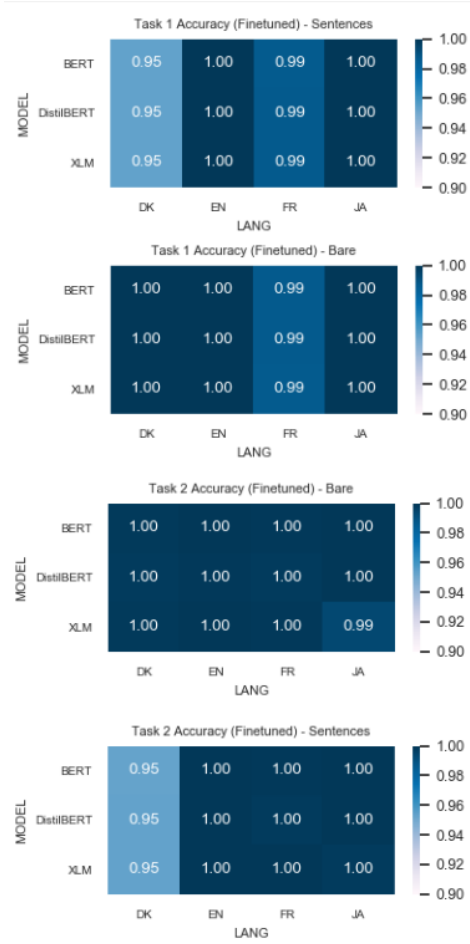

Figure 4: Fine-tuned task validation accuracy per language/model, when trained for 20 epochs.

B Data Generation

B.1 Dataset Parameters

The parameters below were used to generate data for each language per each model per each task variation:

- Data Gen. Seed: 1
- Data Gen. Number Range: [0-999]
- Train Set Size: 30,000
- Validation Set Size: 10,000
- Test Set Size: 10,000
- Shuffle = True

B.2 Task 1 Data

For both variants of task 1 (sentences/bare), grammatical data are generated by creating random numbers then converting them to text through the

num2words (Ogawa) package. For the ungrammatical data, two grammatical numbers are randomly generated, both converted to text, then appended together to create an ungrammatical number. For example the ungrammatical number "fifty-five two hundred" is the combination of "fifty-five" and "two hundred". Another example made from the same original elements could be: "two fifty-five hundred". Grammatical numbers used in splits, however, were only split such that the resulting elements were grammatical words themselves. So, for example, an non-continuous number word string like "fi-tfy te nnine" would never occur.

Since generating numbers with appendage can naturally occur in ungrammatical number words being longer than grammatical, we control for length by limiting our set of ungrammatical number words to words that are *at most* as long as the longest grammatical number. Lastly, we ensure no grammatical numbers are accidentally created in this process by keeping a list of known grammatical numbers in text form (generated by num2words) that ranges from number sufficiently higher than our generation range. For example, if our number word generation ranges from 0-1000, we would make this list of known grammatical numbers from 1-100,000,000. These number words are finally either left bare or inserted into sentences to form our x inputs and are labeled 0 if grammatical and 1 if ungrammatical.

B.3 Task 2 Data

The data for the semantic task are generated by creating pairs of random (grammatical) number words and labeling the pair with one of two categories: 0 if the first numbers is larger than the second and 1 if the second is larger than the first. Through our process of data generation, we ensure that there are never two pairs using the same number. They are then converted to text form for input to a model. These number words are finally either left bare or inserted into sentences to form our x inputs. When numbers are used in sentence templates, it is ensured that the numbers are used in the same template. For example, given the number pair "five" and "six", we could compare the sentences: "There are five apples." and "There are six apples.".

C Modeling

C.1 Pytorch Hugging Face Transformers

We use the configurations below of transformers from Hugging Face Transformers (Wolf et al., 2019) in Pytorch on all of our reported experimental runs. Average runtimes were all around 1 hour or less.

- **DistilBERT**:
  - Class: DistilBertForSequenceClassification
  - Config: distilbert-base-multilingual-cased
  - Tokenizer: DistilBertTokenizer
  - Num. Parameters: 134 million total
- **BERT**
  - Class: BertForSequenceClassification
  - Config: base-multilingual-cased
  - Tokenizer: BertTokenizer
  - Num. Parameters: 110 million total
- **XLM**
  - Class: XLMForSequenceClassification
  - Config: xlm-mlm-100-1280
  - Tokenizer: XLMTokenizer
  - Num. Paremeters: ∼550 million total (inexact)

C.2 Hyperparameters

All experiments which produced our final results shown in the paper were run with the following hyperparemeters which were selected manually by tuning for accuracy over a validation set:

- Epochs: 20 (Range: 10-20)
- Learning Rate: 0.00001 (Range: 1e-5 - 1e-4)
- Minibatch size: 32

C.3 Infrastructure

- GPU: Nvidia Tesla P100
- CUDA Version: 10.1
- Python Version: 3.7

C.4 Code Repository

Our Github repository can be found here. Code is subject to change after publishing of this paper. Refer to the Github README for latest information.